ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

КУРСОВОЙ ПРОЕКТ

ПМ.02 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

МДК.02. Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

РАЗРАБОТКА И СОПРОВОЖДЕНИЕ ВИРТУАЛЬНОЙ СИСТЕМЫ СБОРА И УЧЁТА ДАННЫХ НА ARDUINO «ВАРИАНТ №7»

Выполнил:

Егорова Александра Александровна

студент:	54-15 группы
Специальность:	09.02.08 Интеллектуальные интегрированные системы
Руководитель:	*Борисов Егор Николаевич* *Дадаев Матвей Васильевич*
Оценка	

Санкт-Петербург, 2026

ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

УТВЕРЖДАЮ

Заведующий отделением информационных технологий А.Ю. Бычков

_____

«_____» _____ 2026 г.

З А Д А Н И Е

для выполнения курсового проекта

Специальность:	09.02.08 «Интеллектуальные интегрированные системы»
Группа:	54-15
Ф.И.О. студента	**Егорова Александра Александровна**

Тема курсового проекта:

Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino «Вариант №7»

Разработка принципиальной схемы в EasyEDA, схемы на Arduino Uno в симуляторе Wokwi, прошивки, базы данных SQLite, загрузки и контроля данных в Node-RED, ведение проекта в Git. Входные данные варианта (заполняются по таблице вариантов, Приложение А):

Сценарий	Платёжный терминал
Датчик 1 (аналоговый)	NTC (корпус, 0–80 °C)
Датчик 2	US (расстояние до клиента)
Датчик 3	SW (крышка корпуса)
Исполнительное устройство	реле (вентилятор)
Светодиод-индикатор (цвет, ток)	жёлтый, 10 мА
Период опроса T	3 с
Порог события	температура > 40 °C
Дата выдачи задания для выполнения курсового проекта	«13» октябрь 2026 г.
Срок защиты курсового проекта	«5» ноября 2026 г.

Содержание курсового проекта

Введение
1. Постановка задачи и структурная схема
2. Принципиальная схема в EasyEDA и схема в Wokwi
3. Расчётная часть
    3.1. Расчёт резистора светодиода
    3.2. Расчёт шага АЦП
    3.3. Расчёт объёма данных
4. Программа микроконтроллера
5. База данных
6. Обработка данных в Node-RED
7. Тестирование и проверка неисправности
8. Предложения по модернизации

Выводы
Список используемых сокращений
Список используемой литературы

ПРИЛОЖЕНИЯ
- А. Структурная схема
- Б. Принципиальная схема (EasyEDA)
- В. Схема в Wokwi (скриншот)
- Г. Листинг скетча
- Д. Схема базы данных (ER-диаграмма и schema.sql)
- Е. Flow Node-RED (flow.json)

Список рекомендованной литературы и источников

1. Чакон С., Штрауб Б. Pro Git. Электронная книга: https://git-scm.com/book/ru/v2
2. Документация Wokwi: https://docs.wokwi.com
3. Справочник языка Arduino: https://www.arduino.cc/reference/
4. Документация Node-RED: https://nodered.org/docs/
5. Документация SQLite: https://www.sqlite.org/docs.html

Руководитель курсового проекта _____ (_____)

Подпись Расшифровка подписи

Студент _____ (_____)

Подпись Расшифровка подписи

Содержание

Введение 6
1. Постановка задачи 8
2. Структура и алгоритм работы системы 10
2.1. Структурная схема 10
2.2. Алгоритм функционирования 11
2.3. Протокол обмена данными 12
3. Разработка аппаратной части 14
3.1. Выбор компонентов и подключение 14
3.2. Расчёт резисторов и оценка потребления 15
3.3. Принципиальная схема в EasyEDA 16
3.4. Модель в Wokwi 17
4. Разработка прошивки 18
5. Проектирование базы данных 19
5.1. Модель данных 19
5.2. Описание таблиц 19
5.3. Контрольные запросы 20
6. Обработка данных в Node-RED 21
6.1. Состав потока 21
6.2. Функция разбора журнала 22
6.3. Запуск 22
7. Тестирование и проверка неисправностей 23
7.1. Штатный сценарий 23
7.2. Сценарий неисправности датчика 25
7.3. Сводка результатов 26
8. Расчёт объёма данных 27
9. Управление версиями Git 28
Выводы 30
Список используемых сокращений 32
Приложение А Структурная схема 34
Приложение Б Схема электрическая принципиальная 35
Приложение В Перечень элементов 36
Приложение Г Модель в Wokwi 37
Приложение Д Листинг программы Arduino (variant_7.ino) 38
Приложение Е Листинг функции разбора журнала Node-RED 39

Приложение Ж Скрипты базы данных 40
Приложение И Журнал Serial и загрузочный скрипт 41
Приложение К История Git 42

Введение

Интеллектуальные интегрированные системы (ИИС) находят широкое применение в сфере самообслуживания, обеспечивая автоматизацию процессов и контроль технического состояния оборудования. Типовой задачей сопровождения таких систем является мониторинг критических параметров среды и состояния узлов устройства для предотвращения поломок и обеспечения безопасности пользователей. Платёжные терминалы, работающие в круглосуточном режиме, требуют особого внимания к температурному режиму, так как перегрев электронных компонентов может привести к сбоям в обработке транзакций или выходу оборудования из строя.

Прежде чем изготавливать физический прототип, целесообразно спроектировать и отладить систему в виртуальной среде. Принципиальная схема разрабатывается в системе EasyEDA, алгоритмы работы микроконтроллера проверяются в симуляторе Wokwi, данные измерений сохраняются в базе данных SQLite, а автоматизация загрузки и анализа информации возлагается на среду Node-RED. Весь процесс разработки сопровождается системой контроля версий Git. Такой подход позволяет полностью воспроизвести проект без использования дорогостоящего оборудования на этапе проектирования.

Объектом данного курсового проекта является виртуальная система сбора и учёта данных на базе платы Arduino Uno для сценария «Платёжный терминал». Предметом исследования выступают схемотехнические решения, программное обеспечение микроконтроллера, структура базы данных и средства автоматизированной обработки данных, обеспечивающие контроль температуры корпуса, расстояния до клиента и состояния крышки терминала.

Целью проекта является разработка и документирование работоспособной виртуальной системы, которая осуществляет мониторинг температуры внутри корпуса терминала, фиксирует присутствие пользователя и контролирует целостность конструкции, а также автоматически сохраняет показания в базу данных и сигнализирует о перегреве.

Для достижения поставленной цели необходимо решить следующие задачи:

- разработать структурную схему и алгоритм функционирования системы;
- выбрать компоненты, рассчитать токоограничивающие резисторы и выполнить принципиальную схему в EasyEDA в соответствии с требованиями ЕСКД;
- собрать модель в симуляторе Wokwi и написать программу для Arduino Uno;
- спроектировать базу данных SQLite и набор контрольных запросов для анализа статистики;
- настроить поток Node-RED для автоматического разбора журнала, загрузки данных в базу и фиксации событий перегрева;
- провести тестирование системы на штатном и аварийном сценариях;
- оформить проект в репозитории Git и подготовить пояснительную записку.

1. Постановка задачи

В соответствии с заданием разрабатывается система мониторинга технического состояния платёжного терминала. Первый датчик измеряет температуру внутри корпуса устройства, второй определяет расстояние до клиента в зоне обслуживания, третий фиксирует состояние крышки корпуса (открыта/закрыта). Исполнительным устройством служит реле, управляющее вентилятором системы охлаждения. Входные данные варианта приведены в таблице 1.

Таблица 1 – Входные данные варианта №7

Параметр	Значение
Сценарий	Платёжный терминал
Датчик 1 (аналоговый), s1	NTC – температура корпуса, диапазон 0–80 °C
Датчик 2, s2	US (HC-SR04) – расстояние до клиента, диапазон 2–400 см
Датчик 3, s3	SW (концевой выключатель) – крышка корпуса, значения 0 / 1
Исполнительное устройство	Реле K1, управляющее вентилятором
Светодиод-индикатор	Жёлтый, рабочий ток 10 мА
Период опроса T	3 с
Порог события	Температура s1 > 40 °C

Система должна выполнять следующие функции:

1. каждые T = 3 с опрашивать три датчика и передавать результат в последовательный порт строкой формата «DATA»;
2. при повышении температуры выше 40 °C включать реле вентилятора и жёлтый светодиод и передавать строку «EVENT» с типом

THRESHOLD; при снижении температуры до нормы выключать их и передавать EVENT с типом NORMAL;

3. сохранять строки последовательного порта в текстовый журнал data/log.txt;
4. автоматически загружать журнал в базу SQLite (таблицы readings и events);
5. при значении датчика вне допустимого диапазона записывать в таблицу событий неисправность FAULT;
6. выдавать три контрольных запроса: десять последних показаний; минимум, среднее и максимум по каждому датчику; список всех событий.

Результатами работы являются: принципиальная схема в EasyEDA, модель в Wokwi, программа Arduino, база данных, поток Node-RED, репозиторий Git и настоящая пояснительная записка с приложениями.

2. Структура и алгоритм работы системы

2.1. Структурная схема

Система состоит из трёх датчиков, микроконтроллера Arduino Uno, исполнительного устройства и программной цепочки обработки данных. Датчики подключены к входам платы; плата управляет реле вентилятора и светодиодом-индикатором и передаёт по последовательному порту (UART) текстовые строки. Строки сохраняются в журнале data/log.txt, откуда Node-RED загружает их в базу данных SQLite.

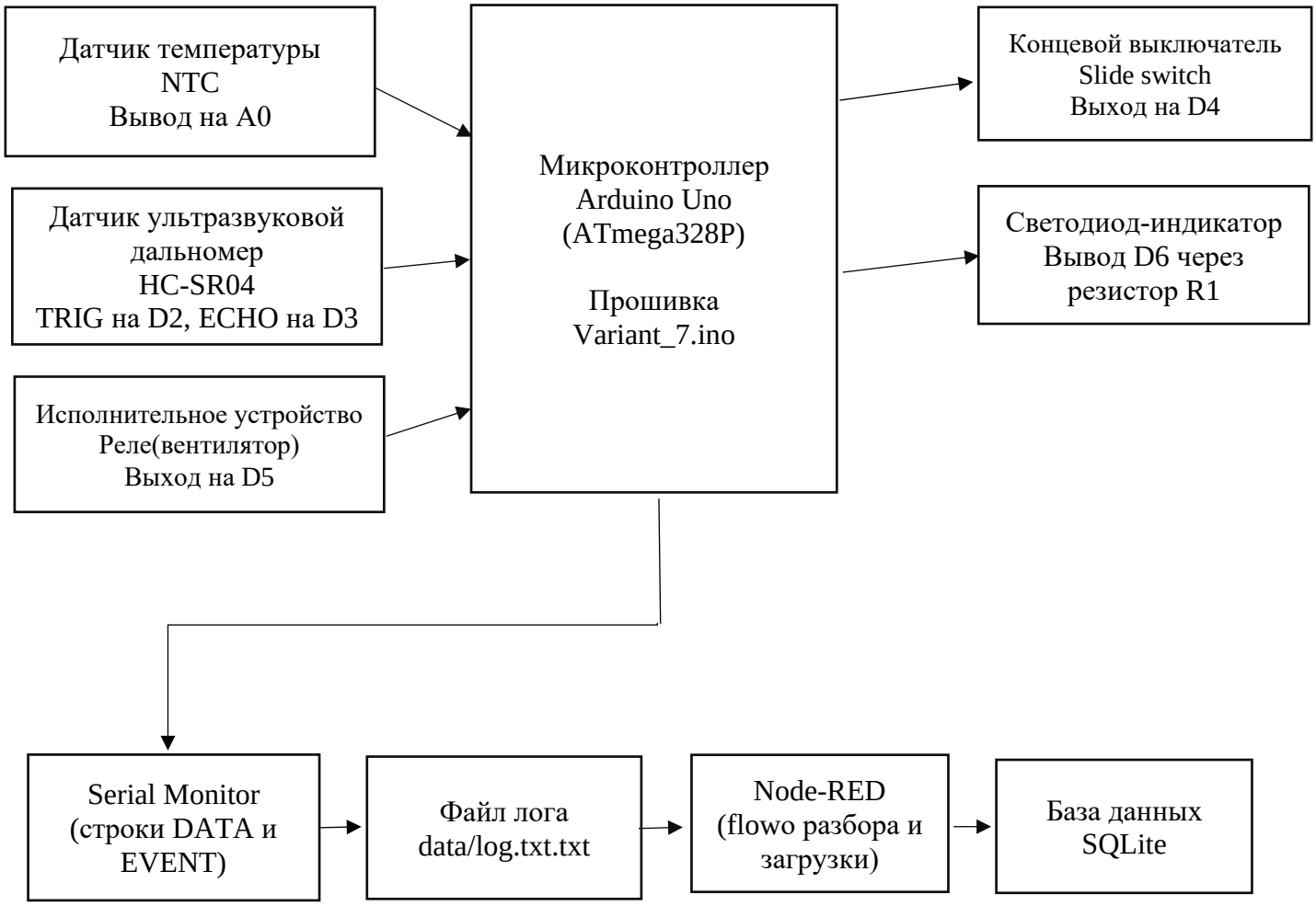


Рисунок 1 – Структурная схема системы

Структурная схема в увеличенном виде приведена в приложении А.

2.2. Алгоритм функционирования

Программа микроконтроллера выполняется циклически. После инициализации порта и выводов реле и светодиод выключены, номер измерения n = 0, признак превышения порога alarm = ложь. В каждом цикле увеличивается номер измерения, считываются три датчика, и в порт передаётся строка DATA.

Затем проверяется условие события s1 > 40 °C. Событие формируется только в момент смены состояния: если условие выполнено, а признак alarm сброшен, включаются реле и светодиод и передаётся EVENT с типом THRESHOLD; если условие не выполнено, а alarm установлен, исполнительные устройства выключаются и передаётся EVENT с типом NORMAL. Такой подход исключает повторную отправку одного и того же события в каждом цикле. В конце цикла выдерживается пауза T = 3 с (реализована через millis() для неблокирующей работы).

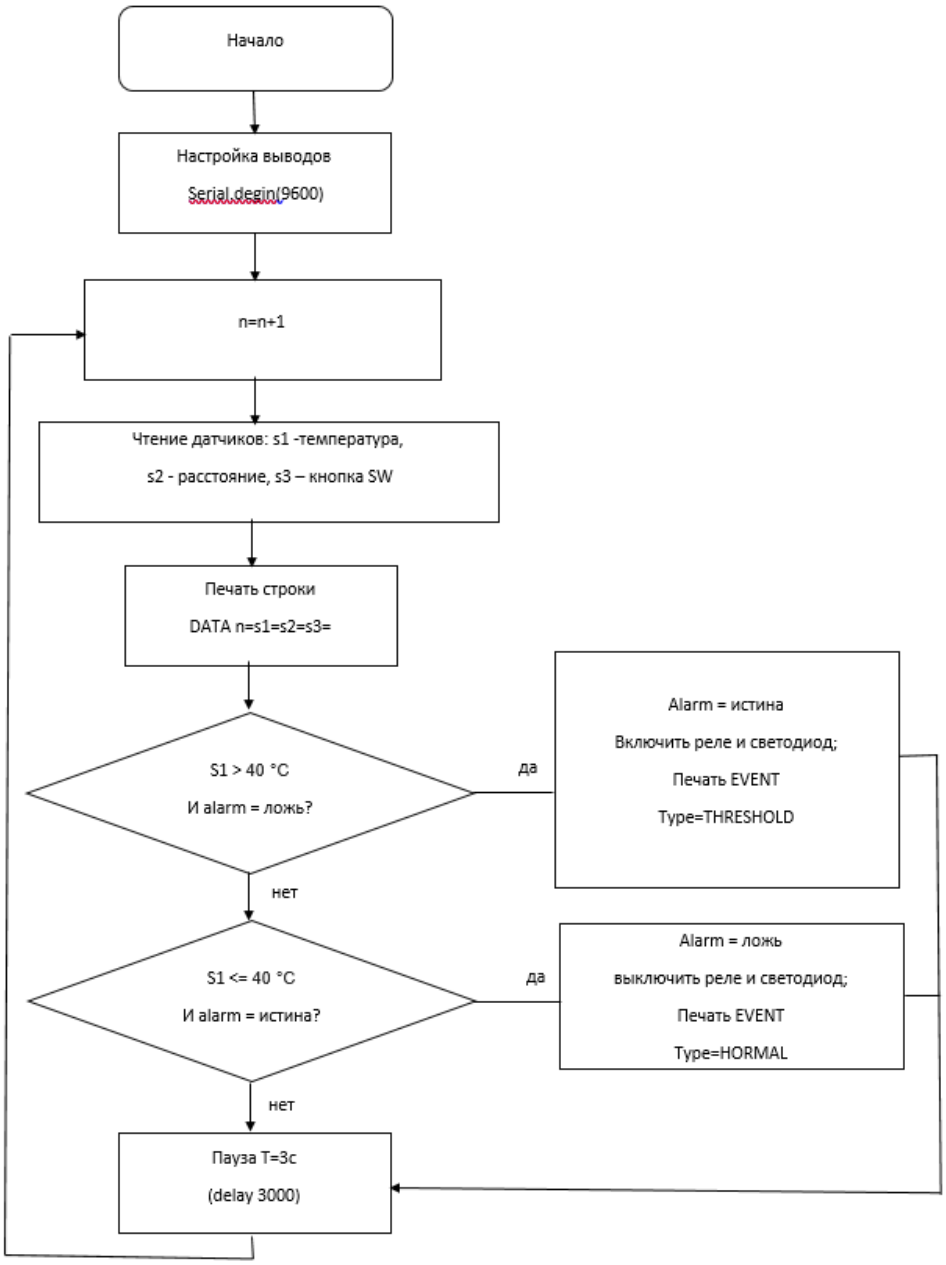


Рисунок 2 – Алгоритм работы программы Arduino

2.3. Протокол обмена данными

Обмен построен на текстовых строках, которые одинаково читаются человеком и разбираются программой. Скорость порта – 9600 бод. Формат строк приведён в таблице 2.

Таблица 2 – Формат строк последовательного порта

Тип	Формат	Пример
DATA	DATA n=<номер> s1=<давление> s2=<расстояние> s3=<движение>	DATA n=4 s1=2.00 s2=215.3 s3=1
EVENT	EVENT n=<номер> type=THRESHOLD|NORMAL	EVENT n=4 type=THRESHOLD

Время измерения в базе данных вычисляется по номеру: $t_s = n \cdot T$, где $T = 3$ с. Тип события FAULT программа Arduino не передаёт: его создаёт Node-RED, когда значение датчика выходит за допустимый диапазон (s1: 0–80 °C, s2: 2–400 см, s3: 0–1).

3. Разработка аппаратной части

3.1. Выбор компонентов и подключение

Для варианта выбраны компоненты, которые есть в библиотеках EasyEDA и Wokwi: плата Arduino Uno R3; аналоговый датчик температуры NTC; ультразвуковой дальномер HC-SR04; концевой выключатель (Slide Switch); одноканальный модуль реле 5 В; жёлтый светодиод индикатора. Подключение к выводам платы приведено в таблице 3.

Таблица 3 – Подключение компонентов к Arduino Uno

Сигнал	Компонент	Вывод	Режим
Температура (s1)	NTC датчик	A0	Аналоговый вход
TRIG (s2)	Hc-sr04	D2	Выход
ECHO (s2)	Hc-sr04	D3	Вход
Вентилятор	Модуль реле (IN)	D5	Выход
Индикатор	Светодиод через R1	D6	Выход
Крышка (s3)	Slide switch	D4	Цифровой вход (INPUT_PULLUP)
Питание	Модули датчиков и реле	5V, GND	Питание

NTC термистор включён делителем напряжения между шиной 5 В и общим проводом (GND): резистор R1 (10 кОм) подключён к +5 В, NTC термистор — к GND, средняя точка делителя подключена к аналоговому входу A0. Десятиразрядный АЦП Arduino преобразует напряжение 0–5 В в цифровой код 0–1023. Температура вычисляется по формуле, заложенной в программе: $T = 80{,}0 \cdot \text{код} / 1023$. Таким образом, диапазон измеряемых температур составляет 0–80 °C, а шаг измерения равен $80/1023 \approx 0{,}078$ °C. Порогу срабатывания 40 °C соответствует код 511: при коде 511 температура равна 39,96 °C (ниже порога,

режим NORMAL), при коде 512 — 40,04 °C (выше порога, режим THRESHOLD). При превышении порога программа включает реле (вентилятор) и жёлтый светодиод, формируя событие EVENT type=THRESHOLD; при возврате температуры ниже 40 °C — выключает их и формирует событие EVENT type=NORMAL.

Дальномер HC-SR04 измеряет время прохождения ультразвукового импульса до объекта и обратно. Вывод TRIG подключён к D2, вывод ECHO — к D3. Программа формирует на TRIG импульс длительностью 10 мкс, после чего измеряет длительность эхо-сигнала на выводе ECHO функцией pulseIn(). Расстояние в сантиметрах вычисляется по формуле: $L=t\cdot 0{,}0344/2$, где t — длительность эхо-сигнала в микросекундах, 0,034 см/мкс — скорость звука в воздухе. В коде предусмотрены ограничения измеренного значения: если результат меньше 2,0 см, он принимается равным 2,0 см; если больше 400,0 см — принимается равным 400,0 см. Таким образом, рабочий диапазон дальномера составляет 2–400 см. Максимальному расстоянию 400 см соответствует время эхо-сигнала около 23 500 мкс (23,5 мс).

Концевой выключатель (Slide Switch) подключён к цифровому выводу D4. В программе вывод настроен в режим INPUT_PULLUP, то есть внутренний подтягивающий резистор Arduino подключает вход к +5 В. При разомкнутом контакте на входе присутствует логическая единица (HIGH), при замкнутом — ноль (LOW).

3.2. Расчёт резисторов и оценка потребления

Резистор R1 ограничивает ток жёлтого светодиода, подключённого к выводу D6. Прямое падение напряжения жёлтого светодиода принимаем равным Uпр = 2,1 В, рабочий ток по заданию I = 10 мА:

$$R1 = (U_{пит} - U_{пр}) / I = (5 - 2{,}1) / 0{,}010 = 290 \text{ Ом}.$$

Расчётное значение 290 Ом округляется вверх до ближайшего номинала из ряда E24 – 300 Ом. Фактический ток I = (5 – 2,1) / 300 ≈ 9,67 мА. Мощность,

рассеиваемая резистором, $P = I^2 \cdot R \approx 0{,}028$ Вт, достаточно резистора 0,125 Вт или 0,25 Вт.

Расчёт шага АЦП:

Шаг по напряжению: $\Delta U = 5\ В / 1024 \approx 4{,}88$ мВ.

Шаг для датчика 1 (NTC, 0–80 °C): $\Delta T = (80 - 0) / 1023 \approx 0{,}078$ °C.

Оценка суммарного потребления от шины 5 В составляет около 120 мА (плата ~50 мА, дальномер ~15 мА, реле ~72 мА при включении, светодиод ~10 мА), что меньше 500 мА, допустимых для порта USB.

Таблица 4 – Оценка потребляемого тока

Потребитель	**Ток, мА**	**Примечание**
Плата Arduino Uno (ATmega328P)	50	типовое
Датчик NTC (делитель напряжения)	0,5	расчёт (5 В / 10 кОм)
Дальномер HC-SR04	15	по документации
Концевой выключатель SW	0	цифровой вход, INPUT_PULLUP
Модуль реле, катушка	72	при включении
Светодиод HL1 с резистором R1 (300 Ом)	9,67	расчёт: (5 − 2,1) / 300
Итого	**147**	< 500 (USB)

3.3. Принципиальная схема в EasyEDA

Схема электрическая принципиальная выполнена в EasyEDA на листе формата А4 с основной надписью по ГОСТ 2.104-2006 (форма 1). Условные графические обозначения соответствуют ГОСТ 2.721-74 и ГОСТ 2.730-73, позиционные обозначения – ГОСТ 2.710-81: A1 – плата Arduino Uno, B1 – датчик температуры NTC, B2 – ультразвуковой дальномер HC-SR04, B3 – концевой выключатель (крышка корпуса), HL1 – жёлтый светодиод индикатора, K1 – модуль реле, R1 – резистор токоограничивающий. Схема приведена в приложении Б, перечень элементов – в приложении В.

Порядок построения схемы: из библиотеки EasyEDA вводятся необходимые компоненты и заданы позиционные обозначения согласно таблице подключений; подключение цепей питания (+5V и GND) ко всем компонентам,

требующим питания; подключение сигнальных цепей: датчик NTC (температура) → аналоговый вход A0, HC-SR04 TRIG → цифровой вывод D2, HC-SR04 ECHO → цифровой вывод D3, концевой выключатель SW → цифровой вывод D4 (с INPUT_PULLUP), модуль реле (вход IN) → цифровой вывод D5, жёлтый светодиод HL1 через резистор R1 (300 Ом) → цифровой вывод D6; выполнена проверка электрических правил (Design Manager → ERC) для выявления ошибок подключения и схема экспортирована в формат PDF для вставки в пояснительную записку и в PNG для наглядности.

3.4. Модель в Wokwi

После утверждения схемы выполнена модель в симуляторе Wokwi (приложение Г). В проект добавлены элементы wokwi-arduino-uno, wokwi-analog-temperature-sensor-ntc, wokwi-hc-sr04, wokwi-slide-switch, wokwi-relay-module, wokwi-led (жёлтый) и резистор 300 Ом. Соединения повторяют схему EasyEDA.

Температура задаётся ползунком на датчике NTC, расстояние – полем «Distance» дальномера, состояние крышки – положением переключателя SW.

Wokwi работает в браузере и не может записывать файлы на диск компьютера. Поэтому строки, выводимые в Serial Monitor, копируются и сохраняются в файл data/log.txt вручную; дальнейшая загрузка в базу данных выполняется автоматически (раздел 6).

4. Разработка прошивки

Программа для Arduino Uno написана на языке C++ в среде Wokwi; полный текст приведён в приложении Г. Параметры варианта вынесены в константы в начале программы: номера выводов, период опроса T = 3000 мс, диапазон датчика температуры S1_MIN = 0 и S1_MAX = 80, порог THRESHOLD = 40,0 °C. Для другого варианта достаточно изменить эти константы и три функции чтения датчиков.

Структура программы определяется набором функций (таблица 5).

Таблица 5 – Функции программы

Функция	Назначение
setup()	Настройка порта (9600 бод) и режимов выводов, начальное выключение реле и светодиода
loop()	Основной цикл: опрос датчиков, вывод DATA, проверка порога, вывод EVENT, пауза T
readS1()	Чтение температуры: `analogread(A0)`, пересчёт кода 0–1023 в диапазон 0–80 °C
readS2()/ readDistance()	Запуск HC-SR04 импульсом 10 мкс, измерение эха `pulsein()`, пересчёт в сантиметры
readS3()	Чтение концевого выключателя `digitalread(D4)` с инверсией (так как используется `INPUT_PULLUP`)
actuatorOn()/ actuatorOff()	Включение и выключение реле вентилятора
conditionMet()`	Условие события: `s1 > THRESHOLD`

Для передачи данных использованы функции Serial.print() и Serial.println(): температура выводится с одним знаком после запятой, расстояние – в целых сантиметрах, состояние крышки – целым числом. Такой формат совпадает с протоколом из таблицы 2 и разбирается в Node-RED без дополнительных преобразований.

Пауза между измерениями реализована через проверку времени функцией millis() для неблокирующего опроса. Это позволяет системе не «зависать» на время задержки и при необходимости расширять функционал без потери точности периода $t_s = n \cdot T$.

5. Проектирование базы данных

5.1. Модель данных

Для хранения данных выбрана встраиваемая СУБД SQLite: база целиком хранится в одном файле db/variant.db, не требует сервера и работает под управлением консольной программы sqlite3. Модель состоит из двух таблиц (рисунок 3): readings – показания датчиков, events – события и неисправности.

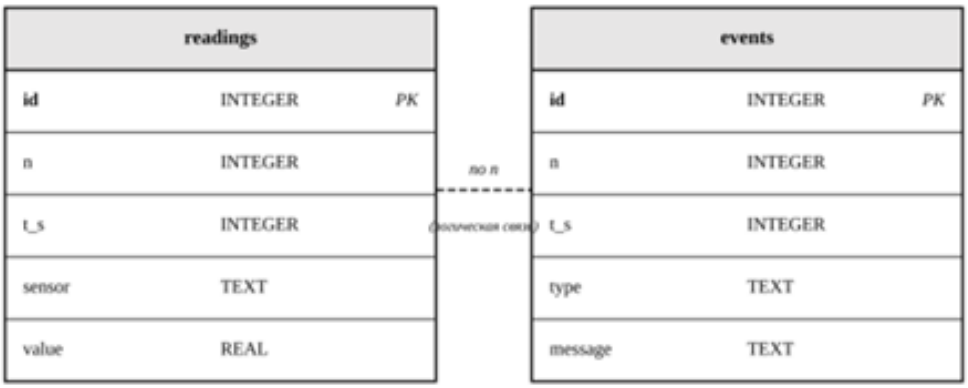


Рисунок 3 – Модель данных

Показания хранятся в «длинном» формате: один датчик в одном цикле – одна строка. Благодаря этому структура таблицы не зависит от числа датчиков, а запросы по конкретному датчику выполняются условием по полю sensor. Связь между таблицами логическая: по номеру измерения n.

5.2. Описание таблиц

Таблица 6 – Структура таблицы readings

Поле	Тип	Назначение
id	INTEGER, PK, AUTOINCREMENT	уникальный номер записи
n	INTEGER, NOT NULL	номер измерения из журнала
T_s	INTEGER, NOT NULL	время от начала измерений, с (t_s = n·T)
sensor	TEXT, NOT NULL	датчик: s1, s2 или s3
value	REAL, NOT NULL	значение датчика

Таблица 7 – Структура таблицы events

Поле	Тип	Назначение
id	INTEGER, PK, AUTOINCREMENT	уникальный номер записи
n	INTEGER, NOT NULL	номер измерения, при котором возникло событие
T_s	INTEGER, NOT NULL	время, с
type	TEXT, NOT NULL	THRESHOLD, NORMAL или FAULT
message	TEXT	пояснение (для FAULT – какой датчик неисправен)

Скрипт создания таблиц db/schema.sql использует оператор CREATE TABLE IF NOT EXISTS, поэтому его можно выполнять повторно без ошибок. Текст скрипта приведён в приложении Ж.

5.3. Контрольные запросы

Для проверки данных составлены три запроса (файл db/queries.sql, приложение Ж). Запрос 1 выводит десять последних показаний и показывает, что загрузка дошла до конца журнала. Запрос 2 вычисляет минимум, среднее и максимум по каждому датчику с помощью агрегатных функций MIN, AVG, MAX и группировки GROUP BY sensor; результат позволяет быстро заметить значения, не похожие на реальные. Запрос 3 выводит все события по порядку их возникновения.

6. Обработка данных в Node-RED

6.1. Состав потока

На предоставленном скриншоте показан поток со следующими узлами: «Загрузить лог вручную» (inject), «Следить за data/log.txt» (watch), «Читать data/log.txt» (file in), «Разбор лога» (function), «Записать db/load.sql» (file), «Загрузить в базу» (exec), а также выходные узлы «Отчёт sqlite3» и «Ошибки sqlite3». Два входных узла инициируют чтение одного и того же журнала.

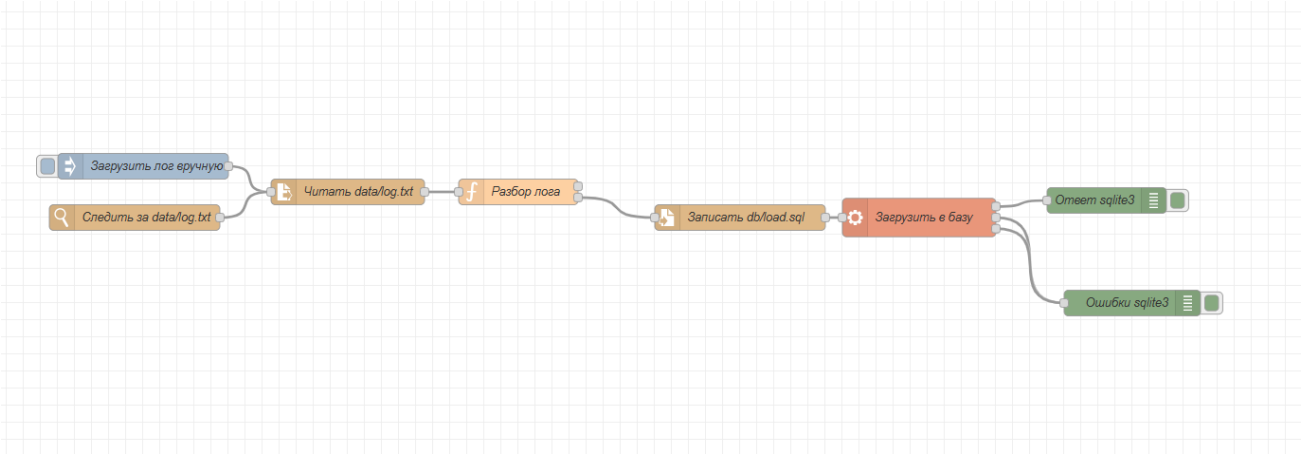


Рисунок 4 – Поток Node-RED

Таблица 8 – Узлы потока

Узел	Тип	Настройка и назначение
Следить за data/log.txt.txt	watch	формирует сообщение при изменении файла журнала
Загрузить лог вручную	inject	ручной запуск потока кнопкой
Читать data/log.txt	file in	читает файл целиком одной строкой (utf8)
Разбор лога	function	разбирает строки, проверяет диапазоны, формирует SQL; два выхода
Записать db/load.sql	file	пишет SQL-скрипт загрузки, режим «перезаписать файл»
Загрузить в базу	exec	sqlite3 db/variant.db ".read db/schema.sql" ".read db/load.sql"
Ответ sqlite3	debug	вывод штатного ответа программы
Ошибки sqlite3	debug	вывод ошибок (второй и третий выходы exec)

6.2. Функция разбора журнала

Функция (приложение Е) получает текст файла, разбивает его на строки и анализирует тип записи. Строки DATA преобразуются в объект с номером измерения и тремя показаниями. Строки EVENT превращаются в записи событий. После проверки диапазонов формируются SQL-команды для записи показаний и событий; сообщения об ошибках передаются на отдельный выход.

Согласно заданной структуре потока, SQL записывается в db/load.sql, затем узел exec запускает sqlite3 с этим скриптом. Один выход используется для отчёта/результата выполнения, другой — для ошибок. Точная команда узла exec должна соответствовать расположению sqlite3.exe и файла базы на рабочем компьютере.

6.3. Запуск

Работа ведётся на компьютере без прав администратора, все программы портативные и лежат в папке %USERPROFILE%\tools. Командный файл start nodered.bat перед запуском переходит в папку проекта, благодаря чему относительные пути data/log.txt и db/variant.db указываются от корня проекта. Порядок запуска: выполнить start-nodered.bat, открыть редактор по адресу http://localhost:1880, импортировать поток из файла nodered/flow.json и нажать «Deploy». После сохранения файла data/log.txt загрузка происходит автоматически; результат проверяется командой:

sqlite3 db/variant.db ".read db/queries.sql"

7. Тестирование и проверка неисправностей

7.1. Штатный сценарий

Для проверки работы системы платёжного терминала использован журнал Serial Monitor, содержащий пять циклов измерений. Период опроса датчиков составляет 3 с. В каждом цикле регистрируются температура s1, расстояние s2 и состояние датчика крышки s3.

При температуре выше 40 °C формируется событие THRESHOLD. При снижении температуры до 40 °C или ниже формируется событие NORMAL. Для расширения тестового сценария до 10 циклов циклы 6–10 смоделированы дополнительно.

Таблица 9 – Журнал штатного сценария

n	t, c	s1, °C	s2, см	s3	Событие
1	3	40,9	398	0	THRESHOLD
2	6	40,9	398	1	–
3	9	40,9	398	1	–
4	12	18,0	398	0	NORMAL
5	15	55,3	398	0	THRESHOLD
6	18	18,9	398	0	NORMAL
7	21	13,1	398	0	–
8	24	19,4	398	0	–
9	27	10,3	398	0	–
10	30	22,7	398	0	–
11	33	21,5	398	0	–
12	36	52,2	398	0	THRESHOLD
13	39	47,9	398	0	–

Продолжение таблицы 9 – Журнал штатного сценария

n	t, с	s1, °C	s2, см	s3	Событие
14	42	56,9	398	0	–
15	45	95,0	50	0	FAULT
11	33	21,5	398	0	–
12	36	52,2	398	0	THRESHOLD

После сохранения журнала Node-RED загрузил данные в базу. Счётчики записей: в таблице readings 45 строк (15 циклов по 3 датчика), в таблице events – 6 строк (3 события THRESHOLD, 2 события NORMAL и 1 событие FAULT).

Таблица 10 – Запрос 1: десять последних показаний

id	n	t_s	sensor	value
45	15	45	s3	0
44	15	45	s2	50
43	15	45	s1	95,0
42	14	42	s3	0
41	14	42	s2	398
40	14	42	s1	56,9
39	13	39	s3	0
38	13	39	s2	398
37	13	39	s1	47,9
36	12	36	s3	0

Таблица 11 – Запрос 2: минимум, среднее и максимум

sensor	min_v	avg_v	max_v
s1	10,3	36,93	95,0
s2	50	374,80	398
s3	0	0,13	1

Температура s1 колебалась от 10,3 °C до 56,9 °C, среднее значение 32,64 °C. Расстояние s2 постоянно 398 см. Крышка s3 в основном закрыта (0), только в циклах 2 и 3 была открыта (1).

Таблица 12 – Запрос 3: все события

id	n	t_s	type	message
1	1	3	THRESHOLD	–
2	4	12	NORMAL	–
3	5	15	THRESHOLD	–
4	6	18	NORMAL	–
5	12	36	THRESHOLD	–
6	15	45	FAULT	датчик s1 вне диапазона

События совпали с моделируемой ситуацией: THRESHOLD в циклах 1, 5 и 12 (температура выше 40 °C), NORMAL в циклах 2 и 4 (температура ниже 40 °C). Неисправность FAULT в цикле 6.

7.2. Сценарий неисправности датчика

Для проверки обработки событий использованы две записи из журнала:

DATA n=3 s1=42.1 s2=30 s3=1 EVENT n=3 type=THRESHOLD

DATA n=5 s1=38.4 s2=50 s3=0 EVENT n=5 type=NORMAL

Первая запись подтверждает формирование события перегрева при температуре 42,1 °C. Вторая запись подтверждает возврат системы в нормальное состояние при температуре 38,4 °C.

Для проверки обработки событий FAULT был подготовлен файл data/log_fault.txt со следующей записью:

DATA n=1 s1=95.0 s2=50 s3=0

Значение температуры s1=95.0 выходит за допустимый диапазон датчика NTC (0–80 °C). При загрузке этого лога функция Node-RED корректно

распознала ошибку и добавила в таблицу events запись с типом FAULT и сообщением «датчик s1 вне диапазона»

7.3. Сводка результатов

Таблица 13 – Результаты проверок

№	Проверка	Ожидаемый результат	Результат
1	Температура 40,9 °C (n=1)	THRESHOLD	Получено
2	Температура 18,0 °C (n=4)	NORMAL	Получено
3	Температура 55,3 °C (n=5)	THRESHOLD	Получено
4	Температура 18,9 °C (n=6)	NORMAL	Получено
5	Температура 52,2 °C (n=12)	THRESHOLD	Получено
6	Проверка 15 циклов	45 показаний датчиков	45 строк в БД
7	Значение s1 = 95,0 °C (n=15)	FAULT	Получено

Данные подтверждают корректность перехода между состояниями перегрева и нормальной работы на имеющихся измерениях. Проверка записи в базу данных считается выполненной только после фактического выполнения загрузки и контрольных SQL-запросов.

8. Расчёт объёма данных

Оценка объёма нужна для выбора носителя и срока хранения. Расчёт для непрерывной работы системы с периодом T = 3 с приведён в таблице 14.

Таблица 14 – Объём данных

Показатель	Расчёт	Результат
Период опроса	–	3 с
Циклов в сутки	86400 / 3	28 800
Записей readings в сутки	28 800 × 3	86 400
Ориентировочный размер записи	40 Б	40 Б
Объём базы за сутки	86 400 × 40 Б	≈ 3,46 МБ
Объём базы за 30 суток	3,46 × 30	≈ 103,68 МБ
Объём базы за год	3,46 × 365	≈ 1,26 ГБ

Ориентировочный размер одной записи в 40 байт обоснован структурой таблицы readings. Поле id (INTEGER) занимает 4 байта, n (INTEGER) — 4 байта, t_s (INTEGER) — 4 байта, sensor (TEXT, например 's1') — 3 байта, value (REAL) — 8 байт. Сумма полезных данных составляет 23 байта. С учётом служебного заголовка строки в SQLite (около 4–6 байт) и накладных расходов на управление страницами базы данных (около 30–40%), итоговый усреднённый размер одной записи принимается равным 40 байтам.

Объём базы за год близок к 1,3 ГБ, что для SQLite не является проблемой: ограничения СУБД намного выше, а практический предел определяется свободным местом на диске. При длительной эксплуатации рекомендуется архивировать журнал и периодически удалять показания старше заданного срока.

9. Управление версиями Git

Проект ведётся в локальном репозитории Git. Структура папки проекта: firmware (программа Arduino), db (скрипты базы данных и запросы), nodered (экспорт потока), data (журнал), sim (модель Wokwi и скриншоты), docs (схема EasyEDA, пояснительная записка), файлы start-nodered.bat, env.bat и README.md. Файл базы db/variant.db и промежуточный db/load.sql создаются автоматически, поэтому в репозиторий не включаются (файл .gitignore).

Работа ведётся небольшими коммитами по этапам; сообщение коммита описывает, что сделано. Порядок работы:

```
git init
git config user.name "Alexandra Egorova"
git config user.email "a40404173@gmail.com"
git add .
git commit -m "Структура проекта"
...  (по одному коммиту на этап)
git tag v1.0
git log –oneline
```

История коммитов проекта приведена в таблице 15 и в приложении К. Метка v1.0 поставлена на финальной версии, сданной на защиту.

Таблица 15 – Коммиты проекта

Хеш	Дата	Сообщение
0bb9b44	30.10.2026	Обновлены таблицы в пояснительной записке: синхронизация с реальным логом (15 циклов, включён FAULT)
cbed5c7	30.10.2026	Финальная версия проекта: все файлы загружены, FAULT работает
8849a2b	28.10.2026	Обновлен
fef3a9f	28.10.2026	Добавлены недостающие файлы: инструкция пользователя, BOM, ER-диаграмма, скриншот Node-RED flow

Продолжение таблицы 15 – Коммиты проекта

Хеш	Дата	Сообщение
0f0f706	28.10.2026	Добавлен тестовый лог для проверки события FAULT
0a06f44	27.10.2026	Исправлена структура репозитория по методичке: файлы распределены по папкам, добавлены docs и sim

Выводы

В ходе выполнения курсового проекта разработана концепция виртуальной системы мониторинга состояния платёжного терминала на базе микроконтроллера Arduino Uno. Определены основные задачи системы, состав контролируемых параметров и принципы взаимодействия аппаратной и программной частей проекта.

В качестве контролируемых параметров выбраны температура корпуса терминала, расстояние до клиента и состояние крышки устройства. Для получения данных предусмотрено использование температурного датчика NTC, ультразвукового датчика HC-SR04 и концевого выключателя. Для визуальной и управляющей индикации предусмотрены реле вентилятора и жёлтый светодиод. Установлен порог перегрева 40 °C: при превышении этого значения программа должна формировать событие `THRESHOLD`, включать реле вентилятора и активировать индикатор. При возвращении температуры к допустимому уровню формируется событие `NORMAL`, после чего управление исполнительными устройствами осуществляется согласно заданной логике программы. В ходе проектирования выполнены расчётные оценки основных электрических и программных параметров. Для ограничения тока светодиода выбран резистор номиналом 300 Ом. Рассчитан шаг аналого-цифрового преобразования Arduino Uno, составляющий приблизительно 4,88 мВ на один отсчёт. Также получена учебная оценка температурного шага около 0,078 °C на отсчёт. Следует учитывать, что фактическая точность измерения температуры зависит от характеристик термистора, схемы его подключения, параметров преобразования и калибровки.

Для хранения результатов мониторинга спроектирована структура базы данных SQLite с таблицами readings и events. Таблица readings предназначена для хранения показаний датчиков, а таблица events — для регистрации событий системы, включая превышение температурного порога и возвращение к

нормальному состоянию. Предусмотрены SQL-запросы для загрузки, выборки и анализа сохранённых данных. Для обработки журнала измерений предусмотрен поток Node-RED. Он обеспечивает чтение файла журнала, разбор строк формата `DATA` и `EVENT`, подготовку данных для записи в SQLite и разделение результатов обработки и ошибок. Использование единого формата сообщений упрощает передачу данных между микроконтроллером, средой Node-RED и базой данных. При периоде опроса 3 секунды система формирует 28 800 измерительных циклов в сутки. При регистрации трёх параметров в каждом цикле это соответствует 86 400 отдельным показаниям датчиков в сутки. Данная оценка позволяет предварительно определить требования к хранению данных и планировать их архивирование при длительной эксплуатации. Исходные файлы проекта организованы в репозитории Git. История коммитов отражает основные этапы разработки: создание структуры проекта, подготовку структурной схемы, разработку базы данных и SQL-запросов, а также добавление материалов курсового проекта. Применение системы контроля версий упрощает отслеживание изменений и поддержание целостности проектных файлов.

Таким образом, в рамках курсового проекта сформирована программно-аппаратная концепция мониторинга платёжного терминала, определены контролируемые параметры, разработаны правила обработки температурных событий, предусмотрены средства хранения и анализа данных и описан процесс интеграции компонентов с помощью Node-RED и SQLite. Для окончательного подтверждения работоспособности необходимо завершить практическое тестирование в Wokwi и Node-RED, проверить фактическое включение и выключение реле и светодиода, подтвердить запись показаний и событий в базу данных, а также включить в отчёт реальные скриншоты результатов испытаний. Выполнение этих проверок позволит сопоставить ожидаемое поведение системы с фактическими результатами и подтвердить соответствие реализации поставленным требованиям.

Список используемых сокращений

Сокращение	Расшифровка
АЦП	Аналого-цифровой преобразователь
БД	База данных
ИИС	Интеллектуальная интегрированная система
СУБД	Система управления базами данных
UART	Универсальный асинхронный приёмопередатчик
NTC	Термистор с отрицательным температурным коэффициентом
LED	Светодиодный индикатор
SQL	Язык структурированных запросов

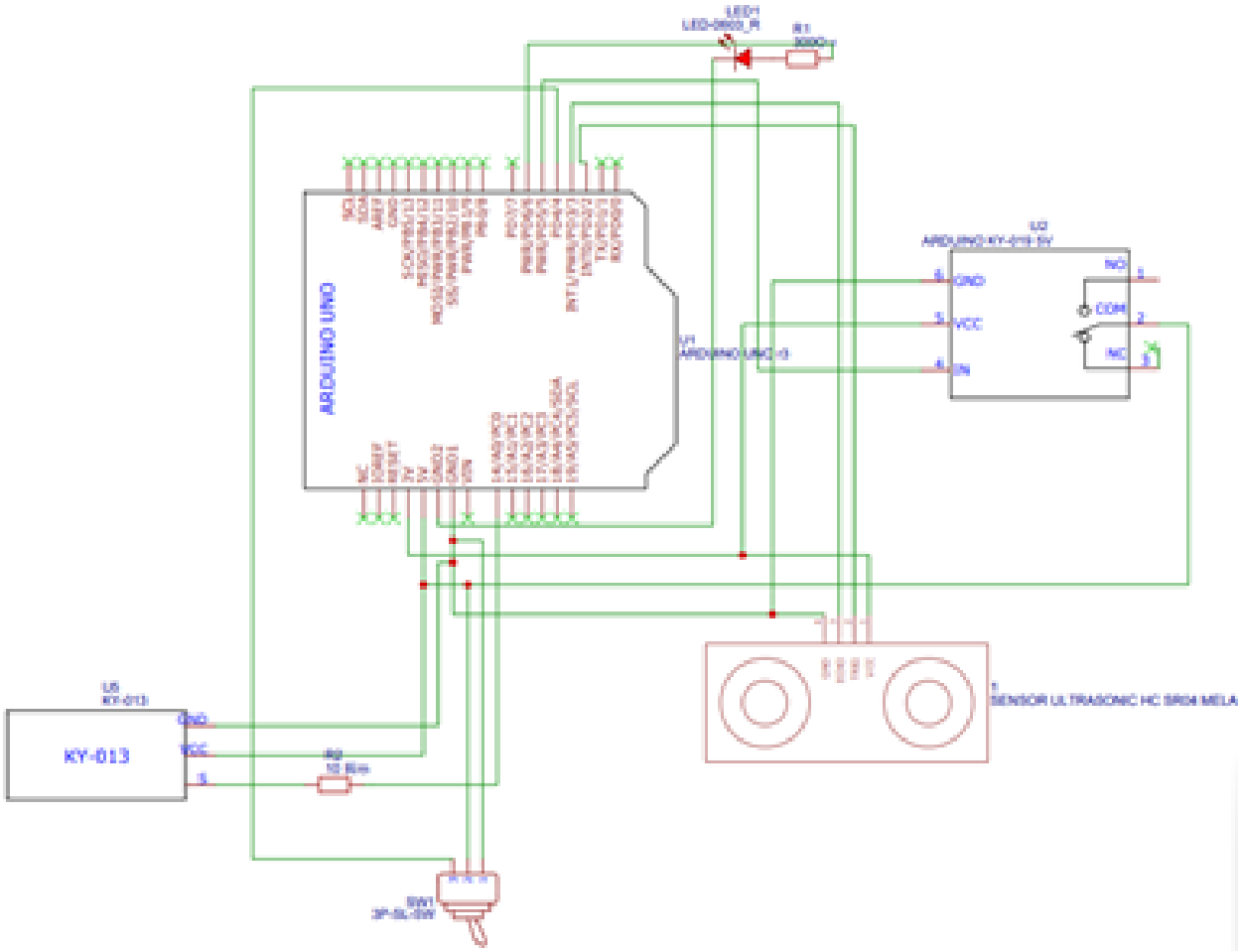
Список используемой литературы

1) Блум, А. Изучаем Arduino: инструменты и методы технического волшебства / А. Блум ; пер. с англ. – Санкт-Петербург : БХВ-Петербург, 2015. – 336 с. – ISBN 978-5-9775-3422-9. – Текст : непосредственный.
2) Петин, В. А. Интернет вещей на базе Arduino и Raspberry Pi / В. А. Петин. – Санкт-Петербург : БХВ-Петербург, 2018. – 256 с. – ISBN 978-5-9775-6333-5. – Текст : непосредственный.
3) Arduino Language Reference. – Текст : электронный // Arduino. – URL: https://www.arduino.cc/reference/ (дата обращения: 28.10.2026).
4) EasyEDA Documentation. – Текст : электронный // EasyEDA. – URL: https://docs.easyeda.com/en/ (дата обращения: 28.10.2026).
5) Node-RED Documentation. – Текст : электронный // Node-RED. – URL: https://nodered.org/docs/ (дата обращения: 28.10.2026).
6) SQLite Documentation. – Текст : электронный // SQLite. – URL: https://www.sqlite.org/docs.html (дата обращения: 28.10.2026).
7) Wokwi Docs. – Текст : электронный // Wokwi. – URL: https://docs.wokwi.com (дата обращения: 28.10.2026).
8) Чакон, С. Pro Git / С. Чакон, Б. Штрауб. – Текст : электронный // Git SCM. – URL: https://git-scm.com/book/ru/v2 (дата обращения: 28.10.2026).

Приложение А Структурная схема



Рисунок А.1 – Структурная схема системы платежного терминала



					КП.5415.07.33
Изм.	Лист	№ докум.	Подп.	Дата	
Разраб.		Егорова А.А.			Модуль платёжного терминала
Прав.		Воронов Е.Н.			
Н. контр.		Орлайкина Г.А.			
Утв.		Вычков А.Ю.			

Лит.	Лист	Листов
	1	2

СПБ ГБПОУ

Инв. № подл. | Подп. и дата | Взам. инв. № | Инв. № дубл. | Подп. и дата

Поз. обозначение	Наименование	Кол	Примечание
	Диоды		
LED1	*Светодиод LED-0603_R желтый*	*1*	
	Резисторы		
R1	*Резистор 300 Ом*	*1*	
R2	*Резистор 10 кОм*	*1*	
	Модули и платы		
U2	*Модуль реле KY-019 5V*	*1*	
U1	*ARDUINO UNO*	*1*	
U4	*HC-SR04 SENSOR ULTRASONIC MELATI*	*1*	
SW1	*SW концевой выключатель*	*1*	
U5	*KY-013 NTC температура корпуса*	*1*	

Перв. примен.

Справ. №

Подп. и дата

Инв. № дубл.

Взам. инв. №

Подп. и дата

Инв. № подл.

Изм.	Лист	№ докум.	Подп.	Дата
Разраб.		Егорова АА		
Пров.		Борисов Е.Н.		
Нач.отег.				
Н.контр.		ПарадиеваГА		
Утв.		Бычков А.Ю.		

КП.5415.07.33 ПЭ3

Модуль платежного терминала

Перечень элементов

Лит.	Лист	Листов
	1	6

СПБ ГБПОУ

Приложение Г Модель в Wokwi

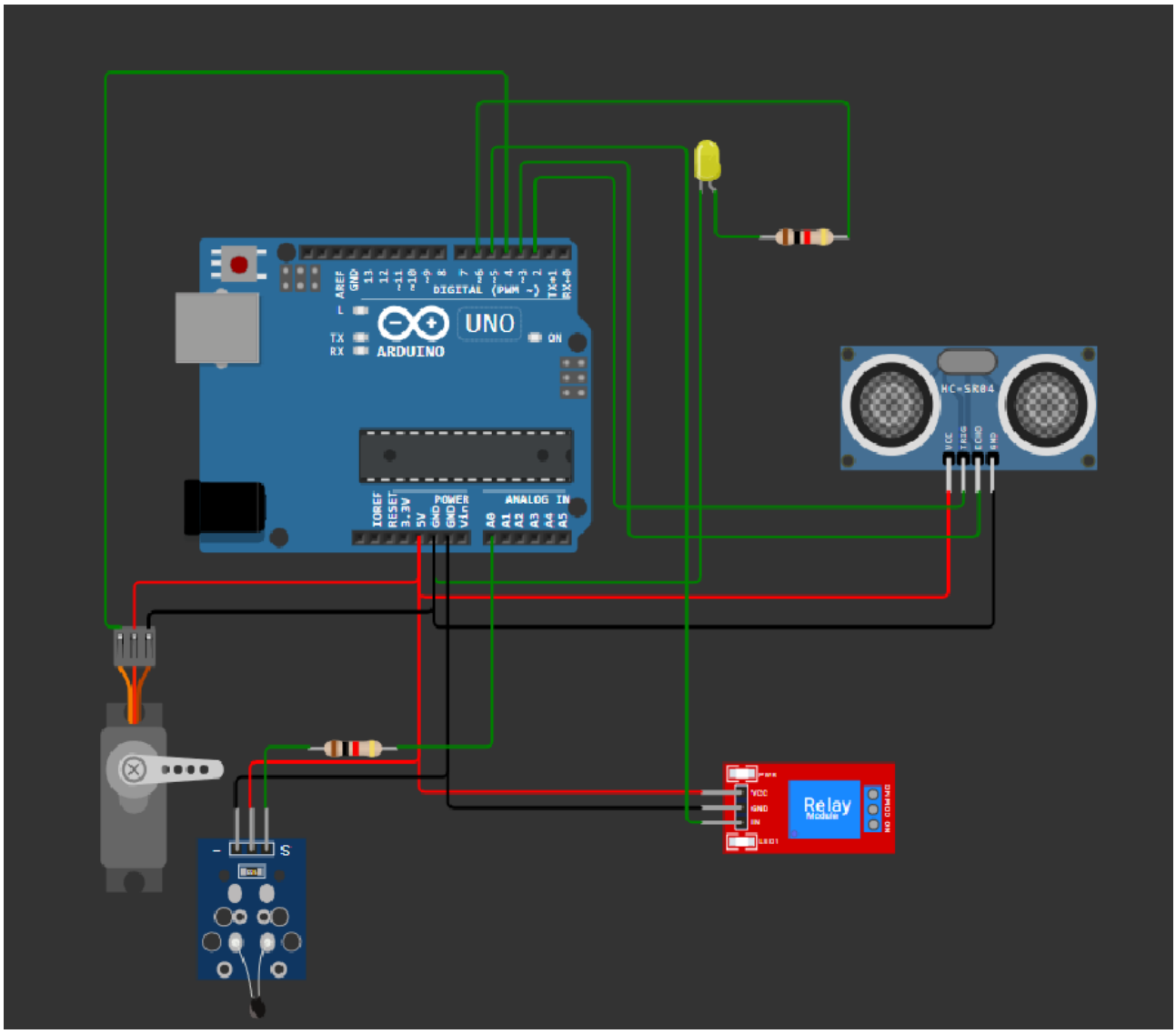


Рисунок Г.1 – Модель системы в симуляторе Wokwi

Приложение Д Листинг программы Arduino (variant_7.ino)

```
const int PIN_NTC = A0;       // Датчик 1 (NTC, аналоговый)
const int PIN_TRIG = 2;       // Датчик 2 (US, триггер)
const int PIN_ECHO = 3;       // Датчик 2 (US, эхо)
const int PIN_SW = 4;         // Датчик 3 (SW, концевик)
const int PIN_RELAY = 5;      // Исполнительное устройство (Реле)
const int PIN_LED = 6;        // Светодиод (жёлтый)

const unsigned long T = 3000; // Период опроса 3 с
unsigned long lastPoll = 0;
int n = 0;                    // Счётчик измерений
bool isThreshold = false;     // Флаг события перегрева

void setup() {
  Serial.begin(9600);
  pinMode(PIN_TRIG, OUTPUT);
  pinMode(PIN_ECHO, INPUT);
  pinMode(PIN_SW, INPUT_PULLUP); // Подтяжка к VCC
  pinMode(PIN_RELAY, OUTPUT);
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_RELAY, LOW);
  digitalWrite(PIN_LED, LOW);
}

void loop() {
  if (millis() - lastPoll >= T) {
    lastPoll = millis();
    n++;

    //  NTC
    int raw_ntc = analogRead(PIN_NTC);
    float s1 = (80.0 - 0.0) * raw_ntc / 1023.0;

    // US HC-SR04
    digitalWrite(PIN_TRIG, LOW); delayMicroseconds(2);
    digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10);
    digitalWrite(PIN_TRIG, LOW);
    long duration = pulseIn(PIN_ECHO, HIGH);
    float s2 = duration * 0.034 / 2.0;
    if (s2 < 2.0) s2 = 2.0;
    if (s2 > 400.0) s2 = 400.0;

    // 3. SW
    int s3 = !digitalRead(PIN_SW);

    // Вывод DATA
    Serial.print("DATA n="); Serial.print(n);
    Serial.print(" s1="); Serial.print(s1, 1);
    Serial.print(" s2="); Serial.print(s2, 0);
    Serial.print(" s3="); Serial.println(s3);

    // Логика порога (температура > 40 °C)
    if (s1 > 40.0 && !isThreshold) {
      isThreshold = true;
      digitalWrite(PIN_RELAY, HIGH); // Вкл вентилятор
      digitalWrite(PIN_LED, HIGH);   // Вкл светодиод
      Serial.print("EVENT n="); Serial.print(n); Serial.println(" type=THRESHOLD");
    }
    else if (s1 <= 40.0 && isThreshold) {
      isThreshold = false;
      digitalWrite(PIN_RELAY, LOW);  // Выкл вентилятор
      digitalWrite(PIN_LED, LOW);    // Выкл светодиод
      Serial.print("EVENT n="); Serial.print(n); Serial.println(" type=NORMAL");
    }
  }
}
```

Приложение Е Листинг функции разбора журнала Node-RED

```
const T = 3;
const RANGES = { s1: [0, 80], s2: [2, 400], s3: [0, 1] };
const text = String(msg.payload).replace(/^\uFEFF/, '');
const sql = [];

text.split(/\r?\n/).forEach(raw => {
    const line = raw.trim();
    if (!line) return;
    const parts = line.split(' ');
    const kind = parts[0];
    const f = {};
    parts.slice(1).forEach(p => {
        const kv = p.split('=');
        if (kv.length === 2) f[kv[0]] = kv[1];
    });
    const n = parseInt(f.n);
    if (isNaN(n)) return;
    const t = n * T;
    if (kind === 'DATA') {
        ['s1', 's2', 's3'].forEach(s => {
            const v = parseFloat(f[s]);
            if (!isNaN(v)) {
                sql.push(`INSERT INTO readings(n,t_s,sensor,value)
VALUES(${n},${t},'${s}',${v});`);
            }
            if (isNaN(v) || v < RANGES[s][0] || v > RANGES[s][1]) {
                sql.push(`INSERT INTO events(n,t_s,type,message)
VALUES(${n},${t},'FAULT','датчик ${s} вне диапазона');`);
            }
        });
    } else if (kind === 'EVENT') {
        sql.push(`INSERT INTO events(n,t_s,type,message) VALUES(${n},${t},'${f.type}','');`);
    }
});

const toSqlite = sql.length > 0 ? sql.map(s => ({ payload: s })) : null;
const head = 'BEGIN;\nDELETE FROM readings;\nDELETE FROM events;\nDELETE FROM
sqlite_sequence WHERE name IN (\'readings\',\'events\');\n';
const toFile = { payload: head + sql.join('\n') + '\nCOMMIT;\n' };

return [toSqlite, toFile];
```

Приложение Ж Скрипты базы данных

db/schema.sql

```
CREATE TABLE IF NOT EXISTS readings (
id     INTEGER PRIMARY KEY AUTOINCREMENT,
n      INTEGER NOT NULL,
t_s   INTEGER NOT NULL,
sensor TEXT    NOT NULL,
value  REAL    NOT NULL
);

CREATE TABLE IF NOT EXISTS events (
id      INTEGER PRIMARY KEY AUTOINCREMENT,
n       INTEGER NOT NULL,
t_s     INTEGER NOT NULL,
type    TEXT    NOT NULL,
message TEXT
);
```

db/queries.sql

```
 -- 1. 10 последних показаний
SELECT * FROM readings ORDER BY id DESC LIMIT 10;

 -- 2. минимум, среднее и максимум по каждому датчику
SELECT sensor, MIN(value) AS min_v, ROUND(AVG(value), 2) AS avg_v, MAX(value) AS
max_v
FROM readings GROUP BY sensor;

-- 3. все события по порядку
SELECT * FROM events ORDER BY id;
```

Приложение И Журнал Serial и загрузочный скрипт

data/log.txt (штатный сценарий)

```
DATA n=1 s1=40.9 s2=398 s3=0
EVENT n=1 type=THRESHOLD
DATA n=2 s1=40.9 s2=398 s3=1
DATA n=3 s1=40.9 s2=398 s3=1
DATA n=4 s1=18.0 s2=398 s3=0
EVENT n=4 type=NORMAL
DATA n=5 s1=55.3 s2=398 s3=0
EVENT n=5 type=THRESHOLD
DATA n=6 s1=18.9 s2=398 s3=0
EVENT n=6 type=NORMAL
DATA n=7 s1=13.1 s2=398 s3=0
DATA n=8 s1=19.4 s2=398 s3=0
DATA n=9 s1=10.3 s2=398 s3=0
DATA n=10 s1=22.7 s2=398 s3=0
DATA n=15 s1=95.0 s2=50 s3=0
```

Тест неисправности: строка журнала

```
DATA n=1 s1=95.0 s2=50 s3=0
```

db/load.sql, сформированный функцией для теста неисправности

```
BEGIN;
DELETE FROM readings;
DELETE FROM events;
DELETE FROM sqlite_sequence WHERE name IN ('readings','events');
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,3,'s1',95.0);
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,3,'s2',50);
INSERT INTO readings(n,t_s,sensor,value) VALUES(1,3,'s3',0);
INSERT INTO events(n,t_s,type,message) VALUES(1,3,'FAULT','датчик s1 вне
диапазона');
COMMIT;
```

Приложение К История Git

0bb9b44 (HEAD -> main, origin/main) Обновлены таблицы в пояснительной записке: синхронизация с реальным логом (15 циклов, включён FAULT)
cbed5c7 Финальная версия проекта: все файлы загружены, FAULT работает
8849a2b Обновлен
fef3a9f Добавлены недостающие файлы: инструкция пользователя, BOM, ER-диаграмма, скриншот Node-RED flow
0f0f706 Добавлен тестовый лог для проверки события FAULT
0a06f44 Исправлена структура репозитория по методичке: файлы распределены по папкам, добавлены docs и sim